NINE CHALLENGES · NINE BREAKTHROUGHS

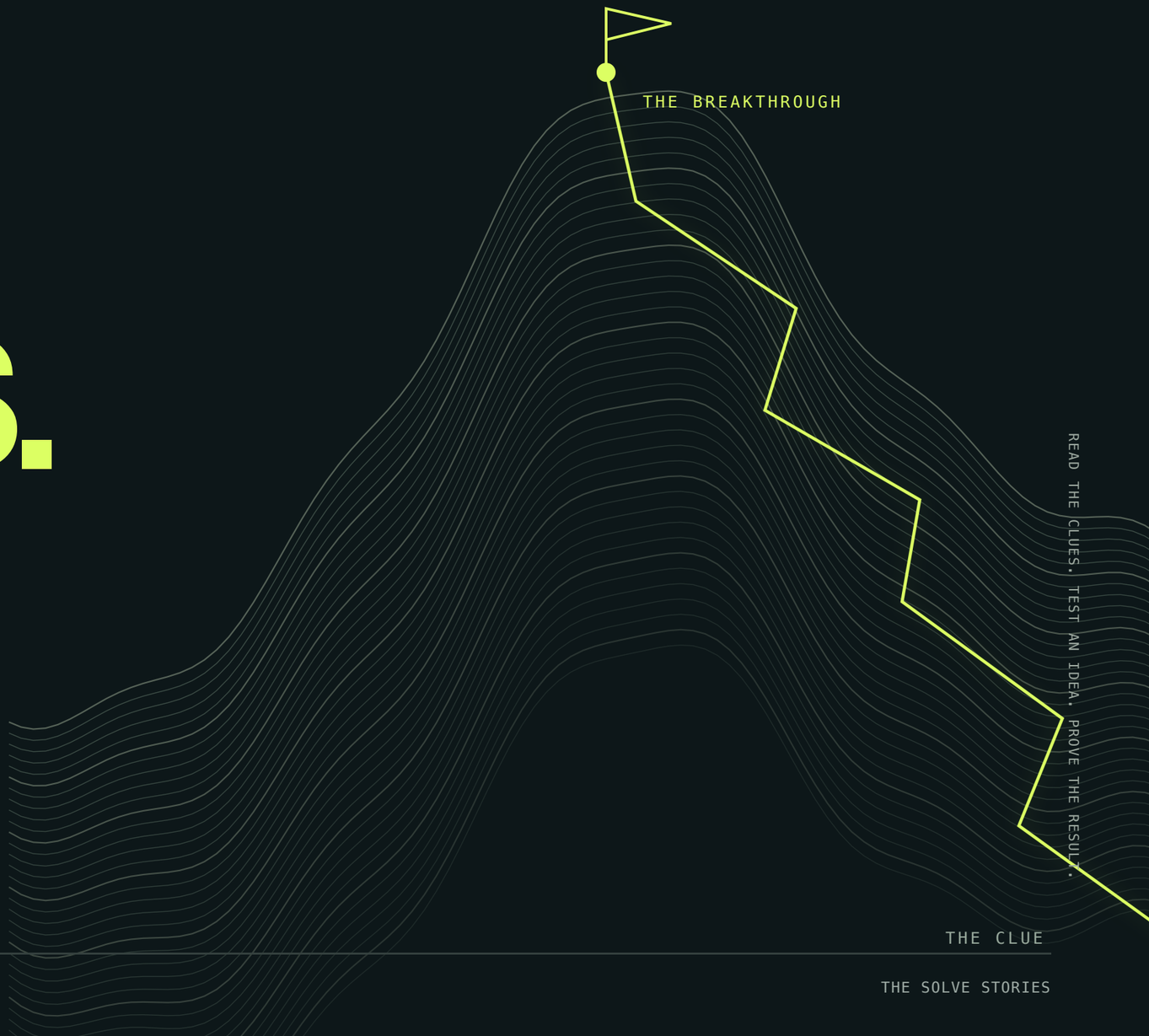
BEHIND THE FLAGS.

The agents found the weaknesses.
Now let’s understand how.

01

LET’S START WITH A SIMPLE IDEA

lost_in_embedding →



THE CHALLENGE · RECOVER HIDDEN TEXT

The flag had disappeared. Only numbers remained.

SUPPLIED FILE

Dockerfile.json

```
model: "gpt2"
inputs_embeds:
  [ 0.1145, -0.3451, 0.2239, … ]
  [ … ]
  …
```

The first values, rounded for display.

24 groups of numbers

768 numbers per group

Each group represents a piece of text, along with information about its position.

THE TAKEAWAY

Our task: **reconstruct the original text.**

UNDERSTAND · WHAT IS AN EMBEDDING?

A piece of text.
A row in a table.

PIECE OF TEXT

BA

A “token”

→

GPT-2’S PUBLIC TABLE

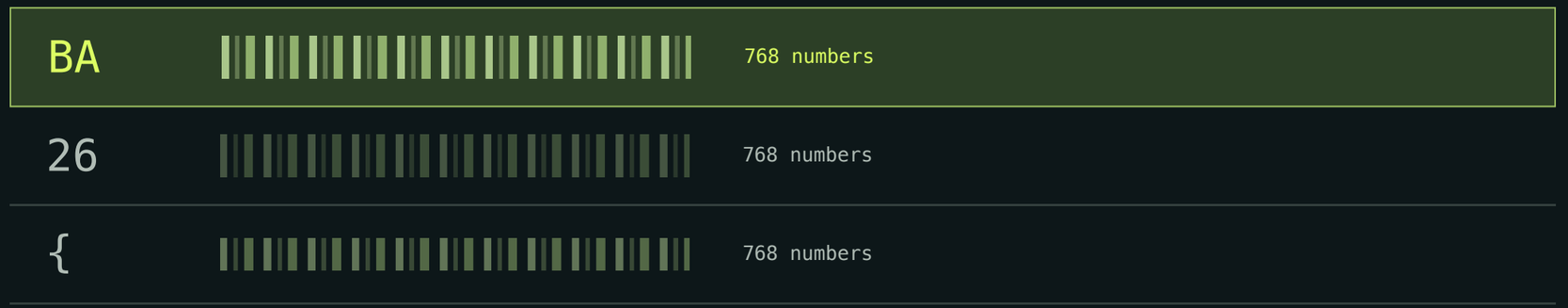


Diagram: the bars represent lists of numbers.

This file also adds: THE TOKEN’S POSITION — To distinguish the first, second, third…

THE TAKEAWAY

The text token and its position come from **two known tables.**

A SMALL EXAMPLE · THREE NUMBERS MAKE IT CLEAR

We know what was added.
So we can subtract it.

A token	[2, 5, 1]	+	[1, 0, 3]	Its position
What we receive	[3, 5, 4]	−	[1, 0, 3]	Known position
What we recover	[2, 5, 1]	→	the token	In the dictionary

An invented 3-dimensional example. In the real file, each row contains 768 numbers.

THE TAKEAWAY

Subtraction works one number at a time.

THE BREAKTHROUGH · WE KNOW BOTH TABLES

Subtract the position.
Recover the token.

01

Read a group

the token’s numbers

+ its position’s numbers

The numbers we received.

02

Subtract the position

the token’s numbers

~~its position’s numbers~~

It is in the public table.

03

Compare with the table

BA ✓

The closest token is found.

THE TAKEAWAY

Repeat for all 24 groups, then join the pieces.

VERIFY · RECOVERING TEXT IS NOT ENOUGH

The text was already readable. The distances confirmed it.

DIRECT COMPARISON

3,5 – 9,9

Distance to the dictionary rows.
The correct tokens already emerge.

AFTER SUBTRACTING THE POSITION

0,000 – 0,004

Almost exact matches.
The proposed structure is confirmed.

24 tokens → The same flag in both attempts ✓

THE TAKEAWAY

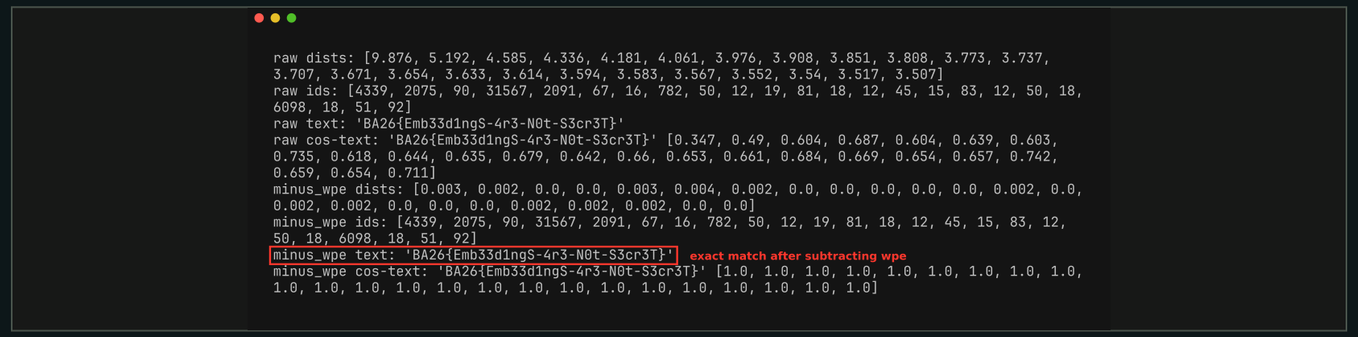
A distance close to zero means: **“we found the same row.”**

THE EVIDENCE · ALL 24 PIECES FIT TOGETHER

The flag put on camouflage. The dictionary recognised it.

BA	26	{	Emb	33	d	1	ng	S	…	}

BA26{Emb33d1ngS-4r3-N0t-S3cr3T}



```
raw dists: [9.876, 5.192, 4.585, 4.336, 4.181, 4.061, 3.976, 3.908, 3.851, 3.808, 3.773, 3.737,
3.707, 3.671, 3.654, 3.633, 3.614, 3.594, 3.583, 3.567, 3.552, 3.54, 3.517, 3.507]
raw ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12, 50, 18,
6098, 18, 51, 92]
raw text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'
raw cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [0.347, 0.49, 0.604, 0.687, 0.604, 0.639, 0.603,
0.735, 0.618, 0.644, 0.635, 0.679, 0.642, 0.66, 0.653, 0.661, 0.684, 0.669, 0.654, 0.657, 0.742,
0.659, 0.654, 0.711]
minus_wpe dists: [0.003, 0.002, 0.0, 0.0, 0.003, 0.004, 0.002, 0.0, 0.0, 0.0, 0.0, 0.0, 0.002, 0.0,
0.002, 0.002, 0.0, 0.0, 0.0, 0.002, 0.002, 0.002, 0.0, 0.0]
minus_wpe ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12,
50, 18, 6098, 18, 51, 92]
minus_wpe text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'   exact match after subtracting wpe
minus_wpe cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

SAVED EVIDENCE · CLICK TO ENLARGE

24 / 24

Tokens recovered.
Almost exact matches
after subtracting the position.

Solved by Claude

THE TAKEAWAY

In this case, turning text into numbers **did not protect its contents.**

THE CHALLENGE · ACCESS THE ADMINISTRATOR ACCOUNT

One admin account. Two ways in.

PASSWORD LOGIN

● ● ● ●

Email + password

The admin’s password remains unknown.

GOOGLE LOGIN

IDENTITY STATEMENT

Account ID

“Here is who is signing in.”

A statement sent to the site

The site reads the account ID.

THE WEAKNESS

On the “legacy” path, **the site did not verify the signature.**

DEAD ENDS · CHANGING THE ROLE IS NOT ENOUGH

Writing “admin”…
The promotion was declined.

	QUESTION TESTED	WHAT THE SERVICE RETURNED
01	Add “role: admin” Can we grant ourselves privileges?	× The account remains a regular user.
02	Keep the email, change the ID Does the site identify an account by its email?	× A different account is created.
03	Keep the same “sub” ID Is this what selects the account?	✓ The same account is found.

`sub` The Google account ID.
The role is then read from the stored account.

THE TAKEAWAY

We needed to find **the real admin account’s ID.**

THE BREAKTHROUGH · THE CLUE WAS IN THE FOOTER

The ID could be found.
The site treated it as a secret.

01

The footer

Tech Corp Chandler

A Gmail contact address

PIED DE PAGE · INDICE REPRÉSENTÉ

A clue recorded in the journal,
in the homepage footer.

02

Looking up the account

Google People API

Using a test Google session

Here, the address leads to an ID.

03

The right ID

1034…4574

GAIA ID / sub · abbreviated

To place in the fabricated statement.

THE TAKEAWAY

The breakthrough: **look up the identity** instead of guessing a password.

THE RESULT · SOLVE RECORDED AT 21:25

The right ID.
An administrator session.

Fabricated identity statement → Existing account found → ADMIN
ROLE OF THE MATCHED ACCOUNT

EXCERPT FROM THE SAVED WRITE-UP · TRANSCRIPTION

socialLogin	**memberId: 1 · role: admin**
/api/flag	**flag returned**

The /api/admin/flag route was a decoy: it returned 404 even when signed in.

`BA26{sub_is_not_a_secret_but_treating_it_like_a_password_is}`

THE TAKEAWAY

Verify **the proof of identity**, don’t just read the ID.

THE CHALLENGE · UNDERSTAND AN UNKNOWN MACHINE

The machine was fluent. We needed the dictionary.

CODES, BUT WHICH ACTIONS?

```
03  →  ?
04  →  ?
19  →  ?
```



A “stack machine”

It stacks values,
then copies them, moves them
or performs calculations.

The helpful clue: 4 sample programs + their expected outputs

THE TAKEAWAY

The examples will help us **recover the meaning of the codes.**

UNDERSTAND · A SMALL EXAMPLE SHOWS THE PRINCIPLE

Copy or swap?
The output tells them apart.

INVENTED TEACHING EXAMPLE · THE REAL TESTS COME NEXT

STARTING STACK

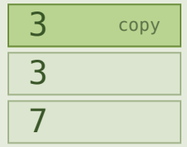


We process the top value first.

Unknown instruction
then print the top

HYPOTHESIS A

Copy the top

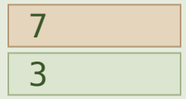


→ OUTPUT 3

✓ consistent

HYPOTHESIS B

Swap the two



→ OUTPUT 7

× rejected

IF THE EXPECTED OUTPUT IS 3 “Swap” cannot be the right instruction.

THE TAKEAWAY

A known example helps us **reject an interpretation that does not fit.**

THE SOLVE · APPLY THIS TEST TO EVERY DICTIONARY

The examples narrow it down. One dictionary remains.

01

First, the stack operations

Try mappings between codes and actions.

Example 1 Example 2

1 mapping

02

Then, the arithmetic operations

Keep mappings that produce the right outputs.

Example 3 Example 4

1 mapping

346 possibilities after example 3 → 1 after example 4

Using the correct interpretation of signed numbers.

THE TAKEAWAY

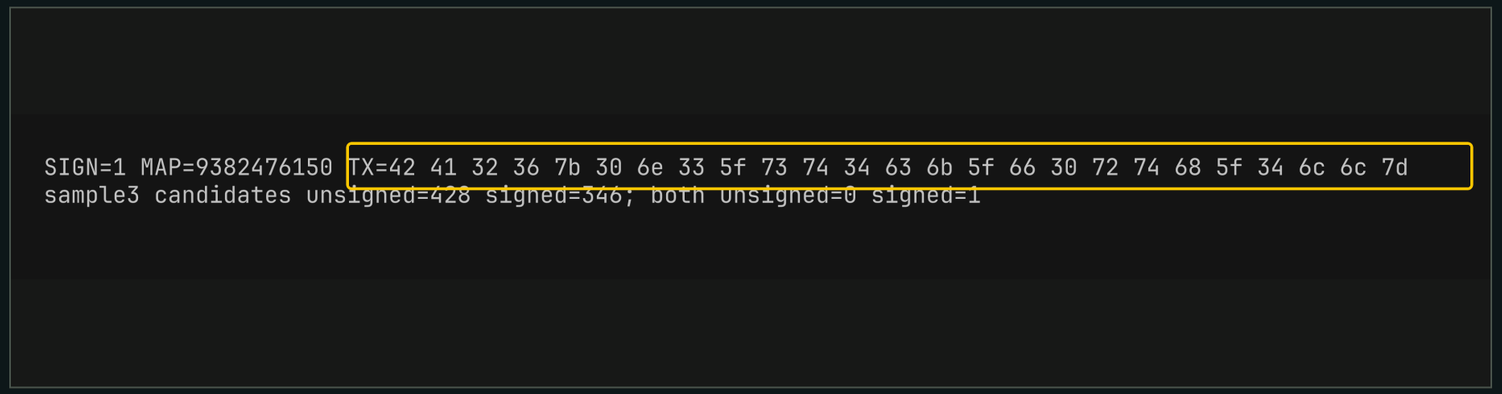
The unknown transmission is read **using this verified dictionary.**

THE EVIDENCE · THE RECOVERED DICTIONARY WORKS

The machine gets its language back. The message becomes readable.

03	copy the top
04	emit a byte
19	add

Three entries from the recovered dictionary.
Codes are written in hexadecimal.



SAVED EVIDENCE · CLICK TO ENLARGE

Solved by: **Codex** · Saved solver output.

`BA26{0n3_st4ck_f0rth_4ll}`

THE TAKEAWAY

The known outputs became **a guide to reading the unknown.**

THE CHALLENGE · OPEN A PROTECTED RECORD

A plaintext flag? Too good to be true.

SUPPLIED FILE

`beacon_flash.bin`

BOOT | PROGRAM | DATA

A copy of the memory
from an ESP32-C6 device.

→

WHAT THE PROGRAM CHECKS

Activation code?

_ _ _ _ _ _ _ _

The correct code allows
the record to be decoded.

THE TRAP ~~BA26{n0t_th15_0n3_k33p_d1gg1ng}~~ Stored in plaintext, but not the result we need.

THE TAKEAWAY

Finding text that looks like a flag **is not enough to prove the solve.**

THE BREAKTHROUGH · RECREATE THE ACTIVATION CHECK

The beacon calculates the code.
The agent repeats the calculation.

TWO READABLE STRINGS IN THE DUMP

`LilyGO T7-C6`

`BA26-BEACON-01`

→

THE PROGRAM TURNS THEM INTO TWO NUMBERS

`56ea3ff6`

`cd1014a9`

Calculated “fingerprints”.

→

COMBINED AS THE PROGRAM DOES

`B80FEF82`

The expected code.

SUPPLIED CODE `?` = ? EXPECTED CODE `B80FEF82` → **If equal: allow decoding.**

THE TAKEAWAY

The calculation and its inputs are present: **the answer can be reconstructed.**

UNDERSTAND · CHECKING ACCESS IS NOT DECODING

The code opens the gate.
Another calculation reads the content.

THE TWO FINGERPRINTS

`56ea3ff6`

`cd1014a9`

Same inputs,
two uses.

01 **Check access**

Calculate the expected code.

`B80FEF82`

02 **Decode the content**

Generate a sequence of numbers.

• • • …

31 BYTES FROM THE DUMP

Encrypted data

with

CALCULATED SEQUENCE

One number per byte

→

BITWISE COMBINATION

`BA26{…}`

THE TAKEAWAY

The solver recreates **the check, then decodes 31 bytes.**

THE EVIDENCE · THE DECODED RECORD

Follow the right branch.
Read the real record.

RECOVERED CODE

B80FEF82 ✓

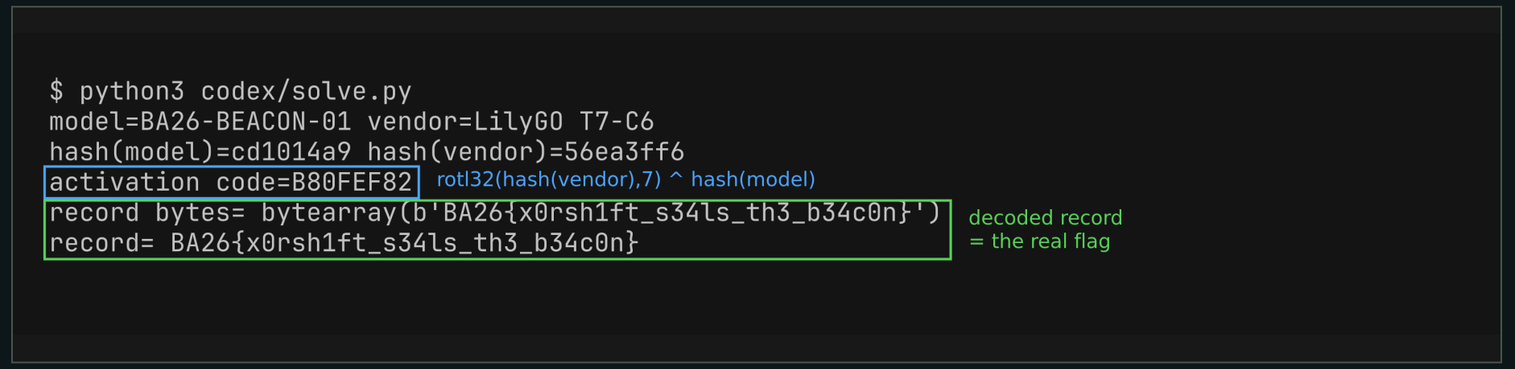
Code calculation

↓

Data decoding

↓

Readable record



SAVED EVIDENCE · CLICK TO ENLARGE

Solved by: **Codex** · Offline solver result.

`BA26{x0rsh1ft_s34ls_th3_b34c0n}`

THE TAKEAWAY

The evidence comes from the **decoding path**, not a string found by chance.

THE CHALLENGE · A FILE STORAGE APP

The server filed things away. We opened the other drawers.

REQUEST RECORDED IN THE JOURNAL	`GET /raw?filename=../../challenge.db`	
EXPECTED DIRECTORY	`/app/uploads/<compte>/`	Files belonging to our account.
CONSTRUCTED PATH	`/app/~~uploads/<compte>/~~../../challenge.db`	Each `../` goes up one directory.
ACTUAL TARGET FILE	`/app/challenge.db`	The account database, outside our directory.

`/download/…` blocks this escape `/raw?filename=…` allows it

Teaching reconstruction. No HTTP capture was preserved.

THE TAKEAWAY

Joining a path **does not guarantee it stays inside the allowed directory.**

THE BREAKTHROUGH · DO NOT STOP AT THE FIRST PLAUSIBLE STRING

A fake flag in the database.
A real lead right beside it.

THE TRAP A password looked like a flag. DECOY

01

Read the accounts

The database also contains an administrator account.

→

02

Log in

The admin dashboard shows more files.

→

03

Follow the clue

`flag.txt` says the flag is hidden in `flag.maze`.

THE TAKEAWAY

The right path leads to a maze made of characters.

THE SOLVER · BREADTH-FIRST SEARCH, OR BFS

Explore the neighbours. Keep track of the way back.

SAVED SOLVER LOGIC · PSEUDOCODE

```
queue ← [entrance]
while the queue contains a cell:
  take the next cell
  add its unvisited open neighbours
  remember their previous cell
```

= wall · **>** = gate · letters lie in the corridors

01 **Explore in layers**

First the neighbours, then their neighbours.

02 **Find the exit**

Each cell remembers where it came from.

03 **Follow the predecessors back**

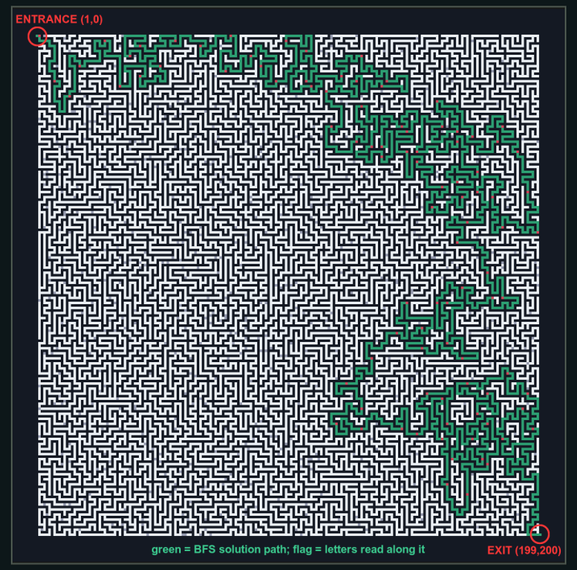
Keep only the entrance → exit path.

THE TAKEAWAY

Read **the letters along this path**, not every letter in the maze.

THE SOLVE · FOLLOW THE MAZE

The path to the exit spells out the flag.



SAVED EVIDENCE · CLICK TO ENLARGE

201 × 201 cells in the maze

1. Start at the entrance.
2. Explore accessible passages.
3. Read the letters on the path to the exit.

2 783 POSITIONS ON THE PATH

`BA26{tHis_Is_a_rEally_loNG_flag_…}`

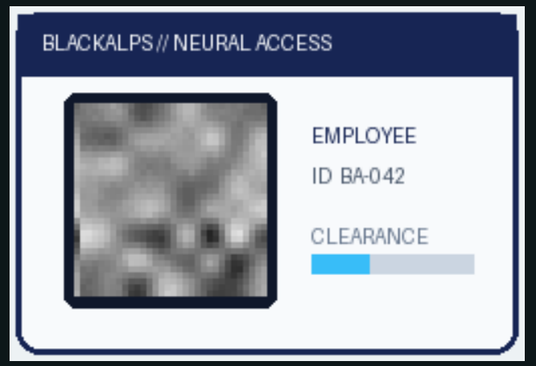
THE TAKEAWAY

File access opens the door. **The maze path gives the answer.**

THE CHALLENGE · AN IMAGE DECIDES ACCESS

To get in as admin, convince the model.

SUPPLIED BADGE



Classified as “EMPLOYEE”.

→

THE SCANNER RETURNS SCORES

EMPLOYEE	
INVALID	
ADMIN	≥ 99.5%

The threshold to exceed for access.

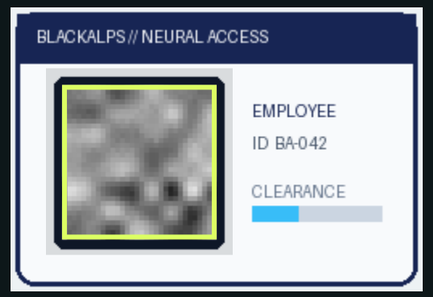
THE TAKEAWAY

We can change the image and observe **the score returned on each attempt.**

THE CLUE · PREPROCESSING IS PROVIDED, THE MODEL STAYS HIDDEN

The model sees just one square. We control every cell.

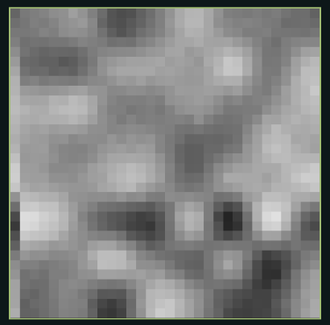
SUPPLIED IMAGE · 256 × 176



The scanner crops the outlined area.

→

SQUARE · 96 × 96



Grayscale, then average each 3 × 3 block.

→

MODEL INPUT · 32 × 32



A block of nine identical pixels sets the value of one cell.

THE TAKEAWAY

Only the cropped square needs changing: **the model never sees the rest of the badge.**

THE BREAKTHROUGH · USE THE MODEL'S FEEDBACK

Test variations in pairs.
Follow the stronger signal.

STARTING POINT The employee badge. Random images were mostly classified as “INVALID”.

01

Vary in both directions

```
badge + variation
badge − variation
```

20 directions tested in pairs:
40 images in one batch.

→

02

Compare the feedback

LESS ADMIN | MORE ADMIN

Score differences
reveal useful directions.

→

03

Update the badge



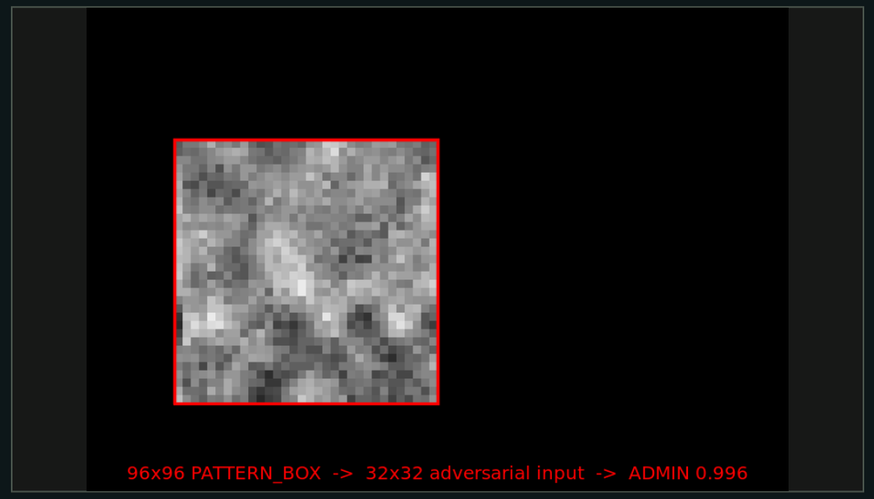
Combine those directions,
take a step, then repeat.

THE TAKEAWAY

The agent learns **how to change the image** by observing the model's response.

THE RESULT · THE MODEL GRANTS ACCESS

The badge looked like noise.
The model saw the boss.



SAVED EVIDENCE · CLICK TO ENLARGE

ADMIN SCORE REPORTED IN THE JOURNAL

99.6%

Required threshold: 99.5%.

ACCESS GRANTED

BA26{r3ached_1h3_t0P}

THE TAKEAWAY

An image-based access decision can be **steered using the exposed scores.**

THE CHALLENGE · RECOVER THE SECRET SENT OVER RADIO

The scale was connected. And a little too chatty.

RADIO RECORDING

10 seconds of signal

Exchanges between the scale and gateway.

ESP32-C6 FIRMWARE

The scale's program

It reveals the protocol and its rules.

LAYER 1 · RADIO

Signal → packets

Recover the transmitted bytes.

LAYER 2 · ZIGBEE NETWORK

Packets → reports

Find the network key in the pairing exchange.

LAYER 3 · APPLICATION

Fragments → flag

Rebuild the key tied to this scale.

THE TAKEAWAY

Three separate steps: **decode, decrypt the network, open the fragments.**

STEP 1 · DECODE THE SIGNAL

Before decrypting, we need to recover the packets.

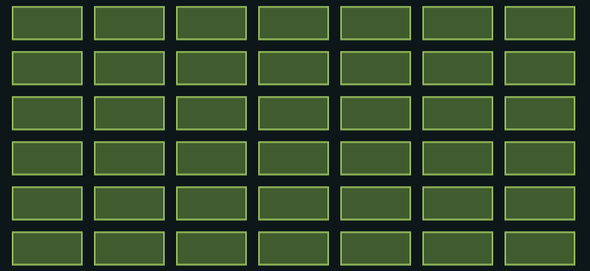
RADIO SIGNAL · ILLUSTRATION



Recognise the patterns used by Zigbee.

→

RECOVERED PACKETS



42 / 42

Valid error checks.

THE TAKEAWAY

The error check confirms **that the bytes were decoded correctly.**

STEP 2 · LISTEN AS THE DEVICE JOINS

When it joins the network, the scale receives a key.

THE PAIRING EXCHANGE IS IN THE CAPTURE

The gateway sends a network key, protected using a known default key.

DEFAULT KEY

`ZigBeeAlliance09`

→

ZIGBEE RULE

Derive a transport key

→

CAPTURED MESSAGE

Open “Transport-Key”

VERIFIED RESULT

Network key recovered ✓

The message authenticity check passes.

→

DECRYPTED REPORTS

6 fragments · 46 bytes

Still encrypted by the application.

THE TAKEAWAY

The network key becomes accessible **through its recorded distribution exchange.**

STEP 3 · TIE THE MESSAGE TO THIS SCALE

The final key depends on the scale's identity.

RADIO IDENTIFIER · OBSERVED EUI-64

40:4c:ca:ff:fe:46:87:c4

+

FIRMWARE CONSTANT

“propolis”

+

FRAGMENT NUMBER

0, 1, 2, 3, 4 or 5

```
[+] network key: b8d8e46f82f99446568d2d30e9512c16
[+] balance EUI-64 (LE on air): c48746feffca4c40  = 40:4c:ca:ff:fe:46:87:c4
[+] fragments: {0: '9f06a697fe855673', 1: '450121232be6ef6f', 2: '72e46c164c1d7da4', 3:
'f23cd1e81057e688', 4: 'd51eb01f236057ca', 5: '73c2ed833409'}
[+] FLAG: BA26{l3s_4b31ll3s_n3_s0nt_j4m41s_s1l3nc13us3s}
```

the recipe = the flag

SAVED EVIDENCE · CLICK TO ENLARGE

Rebuild the key for each fragment.

Apply RC4, then assemble in order.

The identity makes the key device-specific; it is visible in the capture.

BA26{l3s_4b31ll3s_n3_s0nt_j4m41s_s1l3nc13us3s}

THE TAKEAWAY

The solve **rebuilds the keys** from known data, then decrypts with RC4.

THE CHALLENGE · GET ALL 22 DRIVERS TO ACCEPT THE SAME ORDER

The message is simple. Getting it accepted is harder.

TARGET COMMAND **“Box Box Box!”**

A call to pit.

TO CREATE THE PROOF

Private key

Secret. Used to sign.

→

SIGNATURE

Tied to this message

→

TO CHECK THE PROOF

Public key

Known. Used to verify.

THE TAKEAWAY

The public keys are in the binary. **We are missing the private keys.**

THE CLUE · THE TEMPORARY NUMBER USED BY A SIGNATURE

Randomness had 255 options.

255

The “nonce”

A temporary secret number, used for one signature.

> Different from the **private key**, the driver's secret we want to recover.

THE INTEGERS 1 TO 255 · THE ENTIRE SEARCH

001	002	003	004	005	006	007	008	009	010	011	012	013	014	015	016	017
018	019	020	021	022	023	024	025	026	027	028	029	030	031	032	033	034
035	036	037	038	039	040	041	042	043	044	045	046	047	048	049	050	051
052	053	054	055	056	057	058	059	060	061	062	063	064	065	066	067	068
069	070	071	072	073	074	075	076	077	078	079	080	081	082	083	084	085
086	087	088	089	090	091	092	093	094	095	096	097	098	099	100	101	102
103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119
120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136
137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153
154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170
171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187
188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204
205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221
222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238
239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255

OBSERVE	The signature provides a value linked to the nonce.
COMPARE	Try all 255 possibilities and find the matching one.

THE TAKEAWAY

The agent recovers the nonce. **It still needs the signed message.**

THE BREAKTHROUGH · THE MESSAGE IS HIDDEN, ITS FORMATS ARE KNOWN

The radio varied its messages. Not by much.

FIRST APPROACH



Reproduce the randomness from the clock.

~~This approach was later abandoned.~~

THE SUCCESSFUL APPROACH · SOLVER TEMPLATES

MODE	`[pilote] Activate ECU mode {x}`
PLAN	`[pilote] Go plan {lettre}{signe}{tour}`
PIT STOP	`[pilote] Pit window lap {tour}, {pneu}`

A few formats. Small value ranges.

≈ 800

POSSIBLE MESSAGES PER DRIVER

Try each one, instead of decrypting the exchange.

THE TAKEAWAY

The final solution avoids **guessing the timestamp and tracking the random sequence.**

driver_radio/JOURNAL.md: 20:07, 20:12, 20:15; claude/recover.py, the enum() function.

THE METHOD · A HYPOTHESIS MUST PASS A TEST

A candidate key. Then a check.

01
Observed signature
Provided by Spy.

+

02
Recovered nonce
From 255 values.

+

03
Possible message
One hypothesis from the templates.

TOGETHER, THESE LET US CALCULATE

A candidate private key

↓

DOES ITS PUBLIC KEY MATCH THE DRIVER'S?

NO Try another message.

YES The key is confirmed.

22 / 22 Private keys confirmed.
Every candidate is checked.

THE TAKEAWAY

The known public key provides **a confirmation test.**

THE RESULT · SIGN THE TARGET MESSAGE WITH EACH KEY

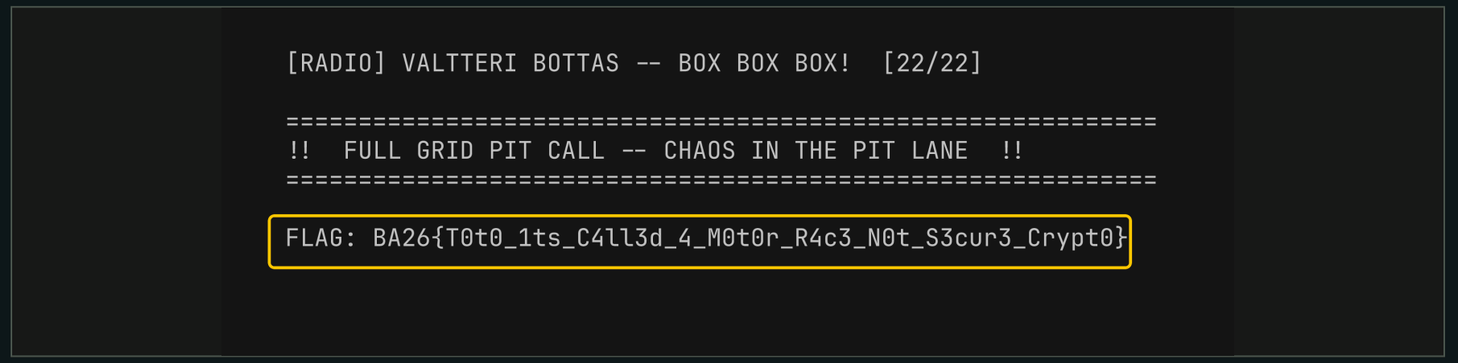
22 drivers in the pits.
So much for the coffee break.

22/22

Commands accepted.

01	02	03	04	05	06	07	08	09	10	11
12	13	14	15	16	17	18	19	20	21	22

ARCHIVED RESULT · NOT A LIVE ATTACK



SAVED EVIDENCE · CLICK TO ENLARGE

The journal and `final_resp.txt` preserve the accepted commands and the flag.

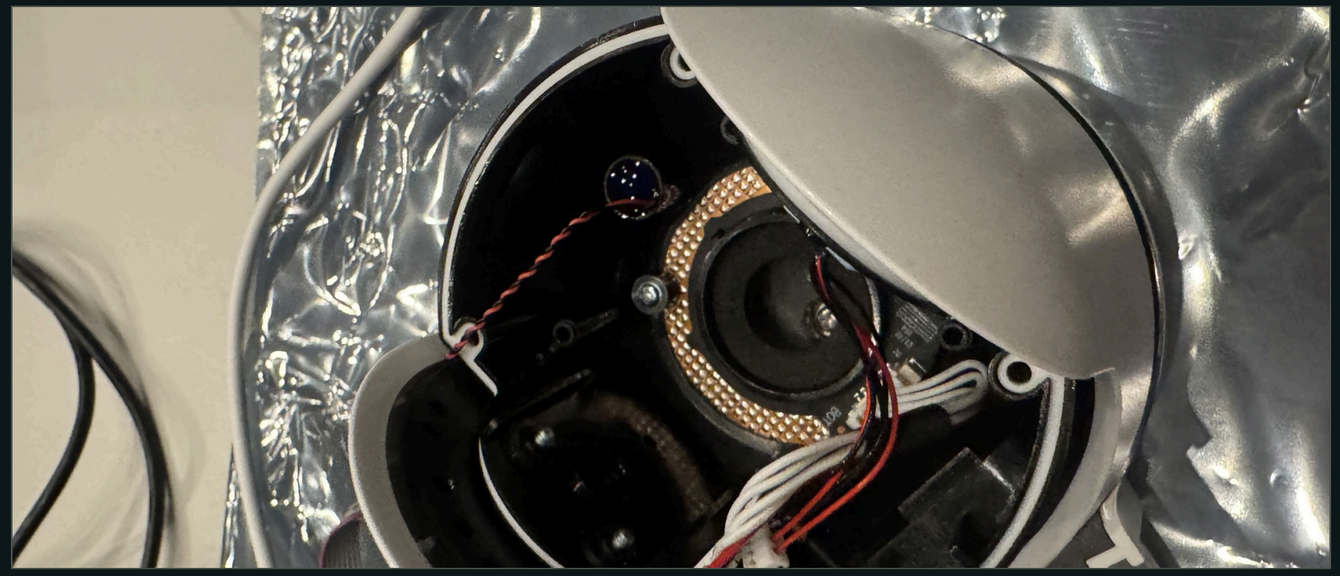
OBSERVE → RECOVER AND VERIFY THE KEYS → SIGN ALL 22 ORDERS

THE TAKEAWAY

The nonce bug and predictable messages made it possible **to recover the signing keys.**

THE CHALLENGE · REAL HARDWARE

It was watching the garden.
We looked inside.



TAPO C500 2.0

A camera.
A small computer.

Its memory holds software,
settings and files.

OUR GOAL

BA26{…}

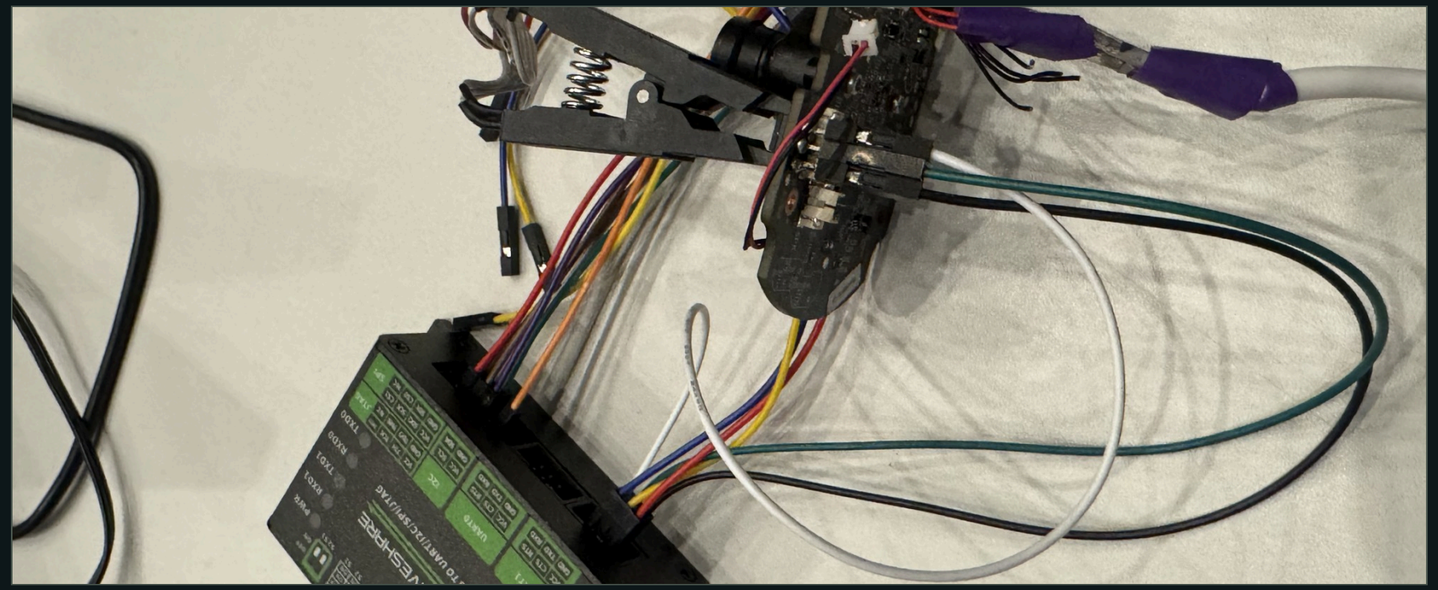
Recover the hidden message.

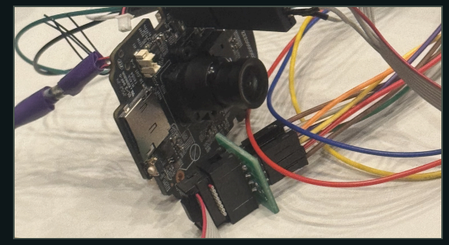
THE TAKEAWAY

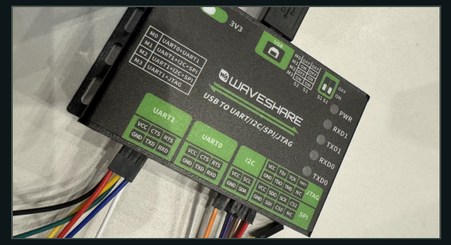
The starting point for analysis: **an 8 MiB memory dump.**

ACQUISITION · COPY THE MEMORY TO STUDY IT

A lot of wires.
One file at the end.







Flash memory

↓ read

Adapter → computer

↓ copy

dump · 8 MiB

THE TAKEAWAY

The investigation continues **inside the recovered file.**

FIRST ATTEMPT · A TEXT SEARCH IS NOT ENOUGH

Ctrl+F found nothing.
We needed a different view.

01 / STORAGE

Compressed files

Take up less space.



→ `files`

Decompress to reach the contents.

02 / MESSAGE

An encrypted flag

Make the text unreadable.

```
9c b6 c8 70 d8 3a…
```

Understand the recipe to recover it.

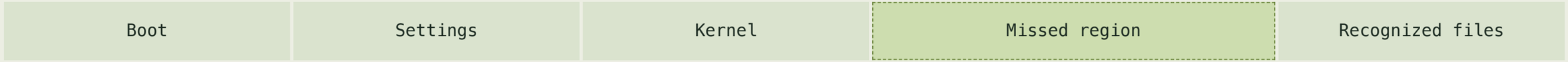
THE TAKEAWAY

This challenge combines file compression and message encryption.

THE BREAKTHROUGH · EXTRACTION HAD MISSED A USEFUL REGION

The book had lost its cover. Not its pages.

DUMP · DIAGRAM NOT TO SCALE



UNRECOGNIZABLE HEADER

?

The first extraction does not see everything.

→

INTACT METADATA

Position	Where to read
Size	How much to read
Fragment	Which block to open

→

RECOVERED FILES

ctf_unlock

flag.enc

Read the right bytes.

THE TAKEAWAY

The tool had missed something. **The memory still had a story to tell.**

THE CLUES · THREE COMPLEMENTARY PIECES

The message was secret.
The instructions were next door.

01

flag.enc

The encrypted message.
35 bytes in the supplied file.

+

02

ctf_unlock

The program that explains how to recover the text.

+

03

devId

The camera's identifier, stored in its settings.

AT STARTUP

```
devId → temporary key → ctf_unlock + flag.enc → /tmp/flag.txt
```

THE TAKEAWAY

Startup connects the clues: **identifier + program + encrypted message.**

THE MECHANISM · REPRODUCE THE RECIPE

Same ingredients.
Exactly the same mask.

`devId · identifier` + `“C500-CTF-v1”` + `counter: 0, then 1`

↓ **SHA-256** One 32-byte hash per counter value.

Block 0 · 32 bytes	Block 1 · 32 bytes

The message is **35 bytes** : use the first block, then 3 bytes of the second.

THE TAKEAWAY

We compute SHA-256 in the usual direction. **We do not reverse it.**

A SHORT PAUSE · THE AUDIENCE CAN DECRYPT TOO

An operation that can go into reverse.

XOR

Two identical bits → **0**

Two different bits → **1**

A four-bit teaching example.
These are not the actual flag bits.

Message

1 0 1 0

Mask

0 1 1 0

Encrypted

1 1 0 0

Encrypted

1 1 0 0

Same mask

0 1 1 0

Message

1 0 1 0

THE TAKEAWAY

Applying **the same mask twice** restores the original message.

FROM DIAGRAM TO CODE · ONE EXCERPT, THREE ACTIONS

The recipe fits in Python. Every line has a role.

01	`key + b'C500-CTF-v1' + struct.pack('<I', counter)`	Assemble the ingredients. The counter takes 0, then 1.
02	`hashlib.sha256(…).digest()`	Produce a 32-byte block. Join the two blocks end to end.
03	`bytes(a ^ b for a, b in zip(encrypted, stream))`	Combine each encrypted byte with its mask byte.

Excerpts from solve.py, separated for readability; "…" abbreviates the ingredients. The script does not execute ctf_unlock.

THE TAKEAWAY

The script connects the extracted files, identifier and recipe. **The result becomes verifiable.**

THE RESULT · TEXT PRESERVED IN FLAG.TXT

Smile.
The flag is out.

EXACT CONTENTS OF THE SUPPLIED FLAG.TXT FILE

```
BA26{a-R00t-0f-Tru5t-Sh1ps-N0-K3ys}
```

35 characters · the same result as in the PowerPoint.

DOCUMENTED STATUS **Flag accepted.** Submission confirmed by the platform.

THE TAKEAWAY

The secret shared its memory with **its recipe and the data needed to read it.**

FROM RESULTS TO REASONING

The flag ends the search.
The evidence tells the story.

01	Identify the gate	What does the system actually check?
02	Find the weak assumption	Is an ID, a score or a list of numbers enough?
03	Confirm the result	Saved output, a key check, an actual flag.

THE TAKEAWAY

The agents found the paths. **Now we can explain them.**